

Réseaux de neurones convolutifs

Application à la base MNIST

Présenté par : Assoulaimani Naima et Darid Khadija

Encadré par : M. Makhlouf Abdenacer

Année universitaire
2025–2026

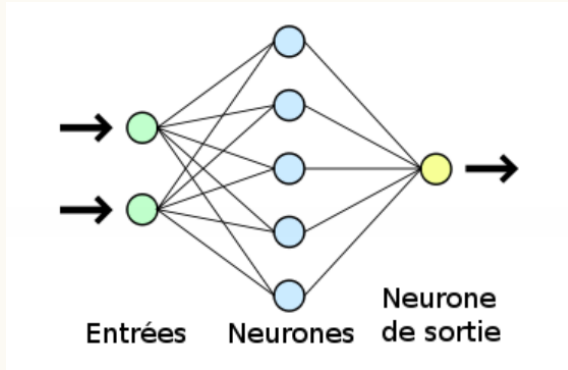
Plan de la présentation

1. Réseaux de neurones artificiels
2. Réseaux de neurones convolutifs
3. Application d'un CNN sur MNIST
4. Conclusion

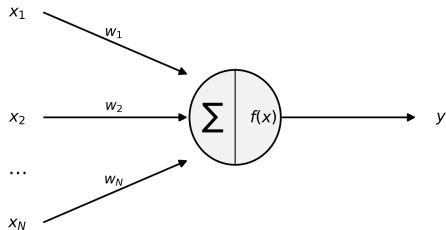


1. Réseaux de neurones artificiels

Réseau de neurones



Entrées → couches de neurones → sortie



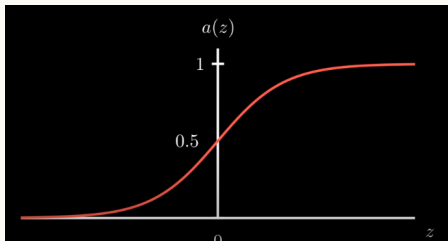
$$f = \omega_1 x_1 + \omega_2 x_2 + \dots + \omega_N x_N + b$$

$$Y = g(f), \quad Y \in \{0, 1\}$$

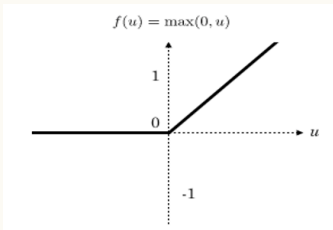
$$g(f) = \begin{cases} 1, & f \geq 0, \\ 0, & f < 0. \end{cases}$$

x_i : entrées ω_i : poids b : biais

Fonctions d'activation

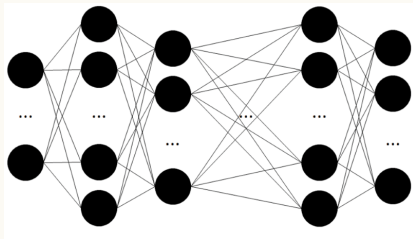


$$\sigma(z) = \frac{1}{1 + e^{-z}}$$



$$\text{ReLU}(z) = \max(0, z)$$

Propagation avant et fonction coût



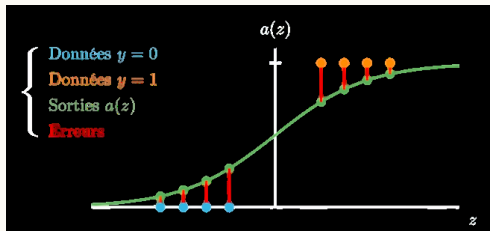
$$X \rightarrow A^{[1]} \rightarrow A^{[2]} \rightarrow \hat{Y}$$

$$Z^{[l]} = W^{[l]}A^{[l-1]} + b^{[l]},$$

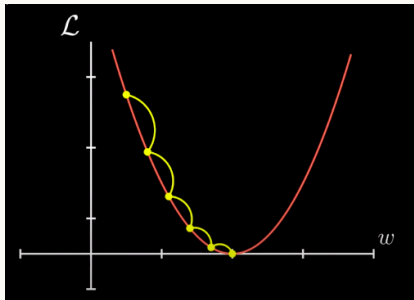
$$A^{[l]} = g^{[l]}(Z^{[l]}).$$

$$L(\hat{Y}, Y) = -\frac{1}{m} \sum_{i=1}^m \ell_i$$

$$\ell_i = y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)$$



Rétropropagation et descente de gradient



$\mathcal{L} \downarrow$

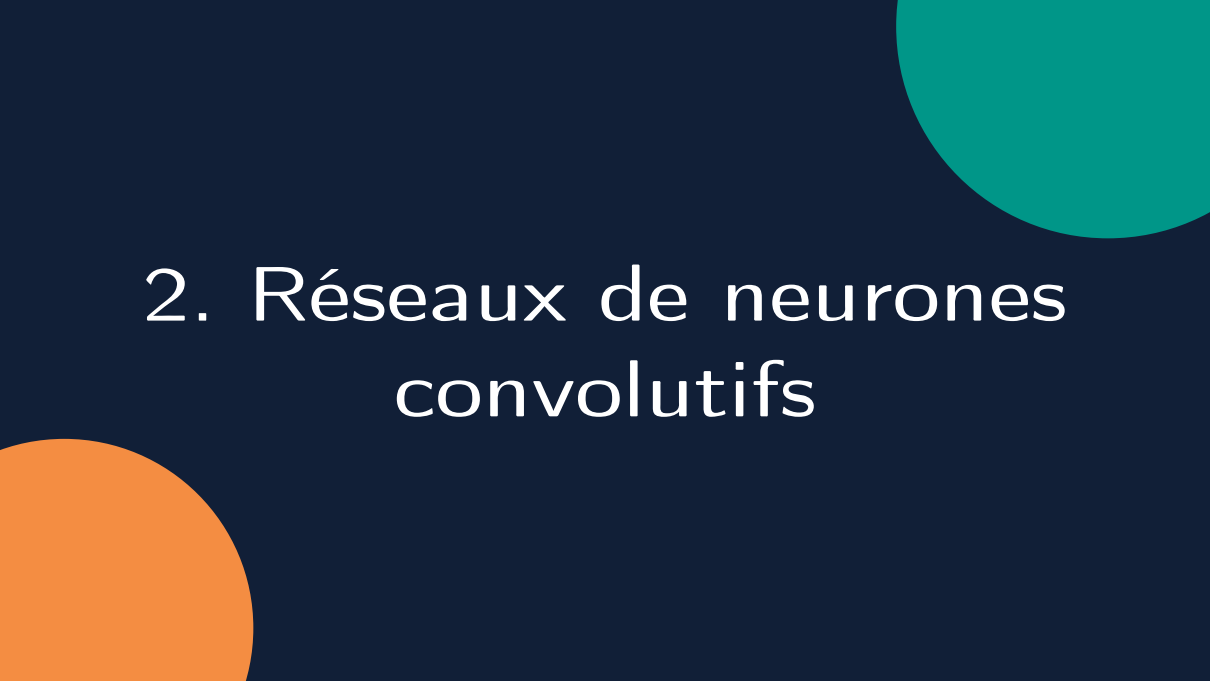
$$\text{erreur} = \hat{Y} - Y$$

$$\frac{\partial \mathcal{L}}{\partial W}, \quad \frac{\partial \mathcal{L}}{\partial b}$$

$$W_{t+1} = W_t - \alpha \frac{\partial \mathcal{L}}{\partial W_t}$$

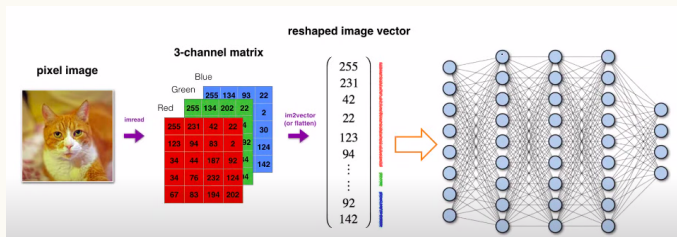
$$b_{t+1} = b_t - \alpha \frac{\partial \mathcal{L}}{\partial b_t}$$

Sortie $\rightarrow$ erreur $\rightarrow$ correction des
paramètres



2. Réseaux de neurones convolutifs

Limites des réseaux classiques pour les images



$$d = 28 \times 28 = 784$$

$$N_{\text{cachée}} = d \times n + n$$

$$28 \times 28 \times 500 + 500 = 392\,500$$

$$500 \times 10 + 10 = 5\,010$$

Image $\rightarrow$ vecteur
structure spatiale perdue
paramètres nombreux

Architecture générale d'un CNN

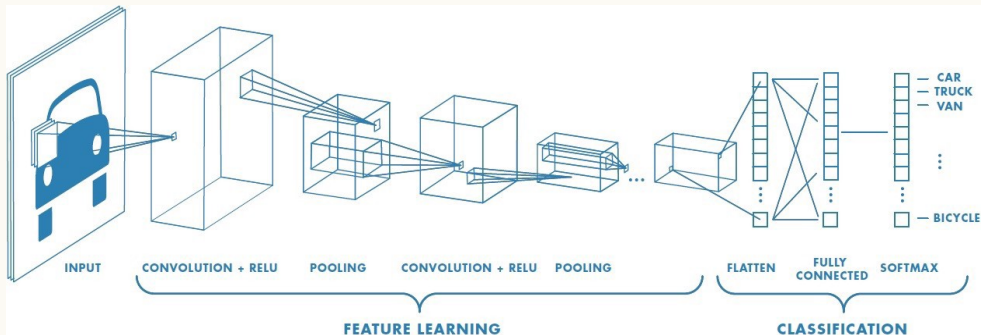


Image → Convolution → ReLU → Pooling → Flatten → Dense → Classe

Convolution 2D

Convolution 2D : calcul local avec un filtre

Image d'entrée $f[i,j]$

12	8	3	5	6
4	9	7	1	2
6	5	10	8	3
1	4	6	9	7
2	3	5	4	8

Voisinage autour du pixel (i,j)

Filtre $h[n,m]$

w1	w2	w3
w4	w5	w6
w7	w8	w9

Multiplication terme à terme
puis addition

Carte de sortie $f'(i,j)$

		S		

La valeur S devient le pixel de sortie

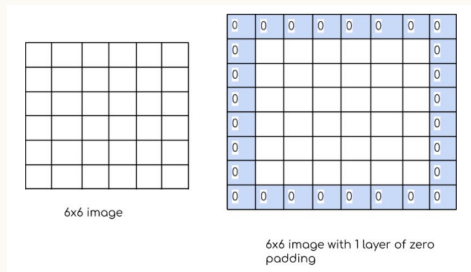
$f'[i,j]$ = somme des produits entre les pixels du voisinage et les coefficients du filtre

$$f'(i,j) = \sum_n \sum_m f[i-n, j-m] h[n,m]$$

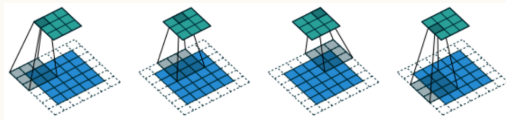
$$\text{activation} = \sum \text{pixels} \times \text{coefficients}$$

Padding et stride

Padding



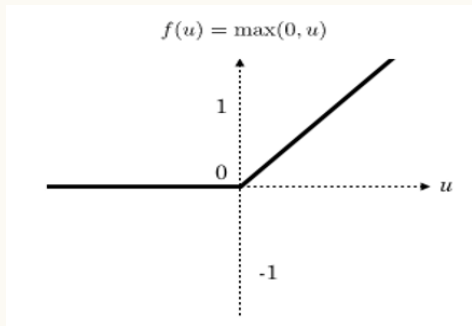
Stride



$$W_{out} = \left\lfloor \frac{W - F + 2P}{S} \right\rfloor + 1$$

$$H_{out} = \left\lfloor \frac{H - F + 2P}{S} \right\rfloor + 1$$

Fonction ReLU

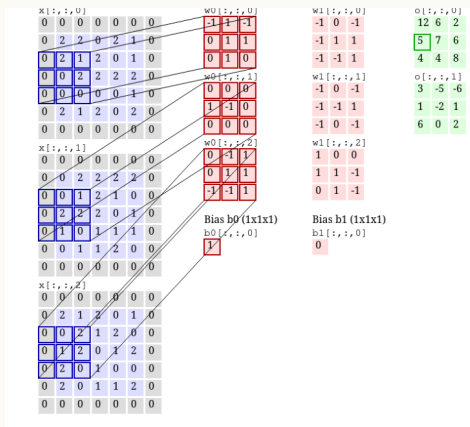


$$\text{ReLU}(x) = \max(0, x)$$

$$\text{ReLU}(x) = \begin{cases} 0, & x < 0, \\ x, & x \geq 0. \end{cases}$$

convolution affine $\rightarrow$ non-linéarité

Couche convolutive complète

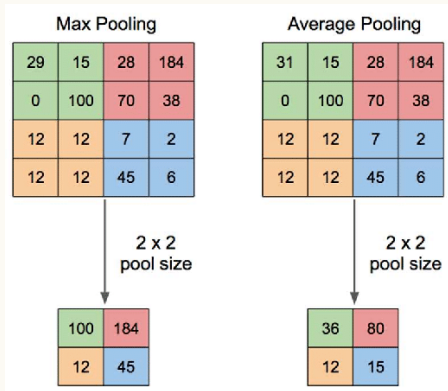


- ▶ Entrée : $7 \times 7 \times 3$
- ▶ Deux filtres : $3 \times 3 \times 3$
- ▶ Sortie : $3 \times 3 \times 2$

$$o_j = \sum_{i=1}^{C_{in}} l_i \star h_{i,j} + b_j$$

$$N_{params} = C_{out} C_{in} k_w k_h + C_{out}$$

Pooling

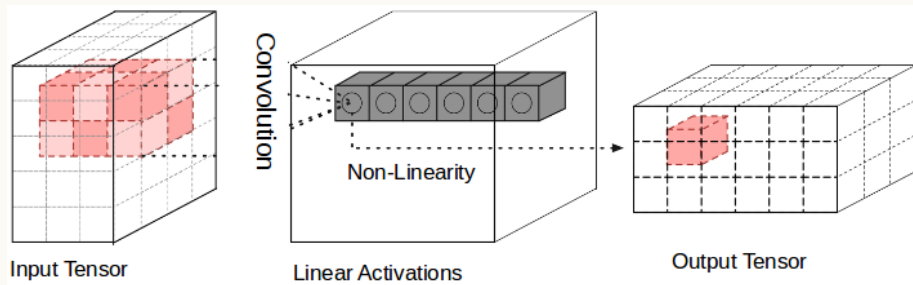


$$\tilde{x} = \max_{i=1, \dots, N} x_i$$

$$\tilde{x} = \frac{1}{N} \sum_{i=1}^N x_i$$

$$(C, W, H) \rightarrow \left(C, \frac{W}{k}, \frac{H}{k}\right)$$

Hiérarchie des convolutions



pixels $\rightarrow$ bords $\rightarrow$ formes simples $\rightarrow$ parties d'objets $\rightarrow$ objet



3. Application CNN sur MNIST

Application : classification des chiffres MNIST

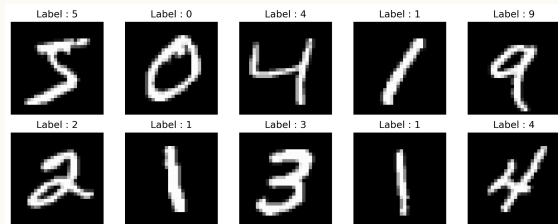
Objectif

Construire un modèle CNN capable de reconnaître automatiquement un chiffre manuscrit entre 0 et 9.

Base utilisée : MNIST

- ▶ 60 000 images d'entraînement ;
- ▶ 10 000 images de test ;
- ▶ images en niveaux de gris ;
- ▶ taille : 28×28 pixels.

Idée : apprendre à associer chaque image à sa bonne classe.



Exemples d'images de la base MNIST

Prétraitement et architecture du CNN

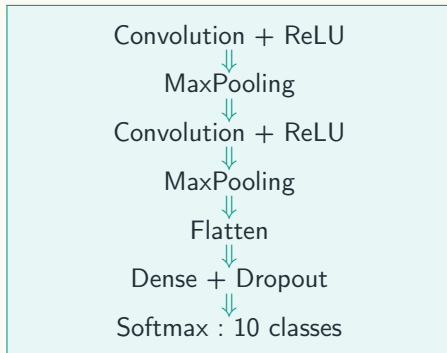
Prétraitement des données

- ▶ normalisation des pixels entre 0 et 1 ;
- ▶ adaptation au format CNN :

$28 \times 28 \times 1$

- ▶ transformation des labels en classes catégorielles.

Architecture utilisée



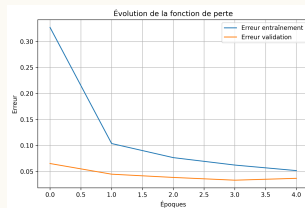
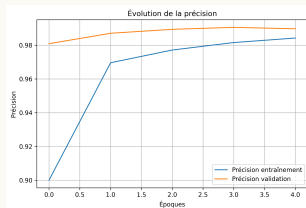
Les convolutions extraient les caractéristiques des chiffres, puis la partie dense réalise la classification.

Compilation et entraînement

Paramètres d'entraînement

- ▶ Optimiseur : **Adam**
- ▶ Perte : **categorical crossentropy**
- ▶ Nombre d'époques : **5**
- ▶ Batch size : **128**
- ▶ Validation : **10%**

Interprétation : pendant l'entraînement, la précision augmente et la perte diminue, ce qui montre que le modèle apprend progressivement.



Évolution de la précision et de la fonction de perte

Évaluation et résultats

Résultat obtenu

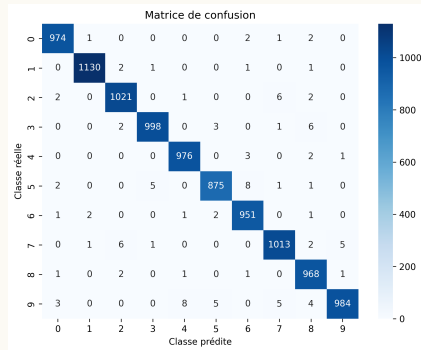
Le modèle classe correctement environ :

9890/10000

images de test, soit une précision
d'environ :

98,90%

- ▶ La majorité des valeurs sont sur la diagonale.
- ▶ Les erreurs restantes concernent surtout des chiffres visuellement proches.
- ▶ Le CNN est donc bien adapté à la reconnaissance de chiffres manuscrits.

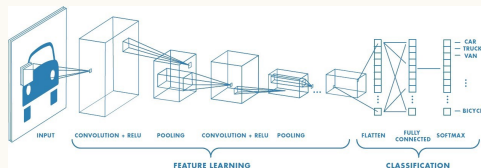


Matrice de confusion
du modèle CNN sur MNIST

4. Conclusion

Conclusion générale

- ▶ Les réseaux classiques donnent les bases de l'apprentissage.
- ▶ Les CNN sont mieux adaptés aux images.
- ▶ La convolution extrait automatiquement les caractéristiques.
- ▶ L'application MNIST confirme l'efficacité du modèle.



CNN $\Rightarrow$ vision par ordinateur

Merci pour votre attention